

Finite-sub-blocklength Latency Minimization for Uplink and Downlink Communications: Theoretical Formulation and Solution Methods

July 6, 2026

Abstract

This report presents the current uplink and downlink work on finite-sub-blocklength latency minimization. It introduces a common notation layer, defines the communication scenarios, derives the finite-sub-blocklength rate expressions used in both links, formulates the uplink and downlink latency-minimization problems, and then develops two solution frameworks: the *Training Only Method* and the *Training + Testing Method*.

1 Introduction

Short-packet communication cannot be analyzed accurately through asymptotic capacity alone. When the codeword length is finite, the achievable rate depends not only on channel quality and transmit power, but also on sub-blocklength and the required decoding reliability. For latency-sensitive multiuser communication, this creates a coupled design problem: the number of transmitted bits, the chosen sub-blocklength, and the precoder must be selected jointly.

The present work studies this problem in both uplink and downlink settings. In the uplink, each user is handled through a user-wise sequence of transmission blocks, and the main difficulty is that the number of required blocks is not known a priori. In the downlink, all active users share the same transmission epoch, so the main difficulty is inter-user interference and the resulting coupling of feasible rates.

The report

1. establish a common mathematical notation for uplink and downlink,
2. define the communication scenarios that determine what each block is asked to deliver,
3. derive the corresponding finite-sub-blocklength uplink and downlink formulations, and
4. present the *Training Only Method* and the *Training + Testing Method* in algorithmic form.

2 System Model and Common Notation

2.1 Core Variables

Table 1 lists the symbols that are shared throughout the report.

The latency of user k is defined by

$$\tau_k = \frac{1}{f_{s,k}} \sum_{\ell} n_{k,\ell}, \quad (1)$$

and the network objective is to minimize the aggregate latency

$$\tau_{\text{sum}} = \sum_{k \in \mathcal{K}} \tau_k. \quad (2)$$

Table 1: Common notation

Symbol	Meaning
$\mathcal{K} = \{1, 2, \dots, K\}$	Set of users.
$k \in \mathcal{K}$	User index.
ℓ	Transmission-block index. In uplink it is per user; in downlink it is per Base Station.
B_k	Total payload of user k in bits.
$b_{k,\ell}^{\text{req}}$	Requested bits to send for user k in block ℓ . In payload completion, $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$; in fixed continuous communication, $b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}$.
$b_{k,\ell}$	Bits actually sent to user k in block ℓ .
$B_{k,\text{rem}}^{(\ell)}$	Remaining unsent bits of user k when block ℓ starts in payload-completion mode.
$\bar{b}_{k,\ell}$	Target bits to send for user k in block ℓ in fixed continuous communication mode.
$n_{k,\ell}$	sub-blocklength assigned to user k in block ℓ , measured in channel uses.
T_k	Maximum admissible sub-blocklength for user k within one coherence-limited transmission opportunity.
$f_{s,k}$	Symbol rate of user k in symbols per second.
τ_k	End-to-end latency of user k .
ε_k	Target packet error probability of user k .
P_k	Uplink power budget of user k .
P_B	Total transmit-power budget of the downlink Base Station Precoder.
d_k	Number of transmitted spatial streams of user k .
$\mathcal{A}_\ell$	Active-user set in downlink block ℓ .
q	One query, i.e. one visited state presented to the learned model.
$\mathcal{Q}$	Collection of training queries used in the Training + Testing Method.

2.2 Finite-sub-blocklength Template

For both uplink and downlink, the achievable rate is described through the normal approximation [1]. Once the specific matrix $\mathbf{A}_{k,\ell}^{(m)}$ has been defined, the corresponding capacity term is

$$C_{k,\ell}^{(m)} = \log_2 \det(\mathbf{I} + \mathbf{A}_{k,\ell}^{(m)}), \quad (3)$$

and the channel-dispersion term is

$$V_{k,\ell}^{(m)} = (\log_2 e)^2 \text{tr} \left[\mathbf{A}_{k,\ell}^{(m)} (\mathbf{A}_{k,\ell}^{(m)} + 2\mathbf{I}) (\mathbf{A}_{k,\ell}^{(m)} + \mathbf{I})^{-2} \right]. \quad (4)$$

Therefore, for sub-blocklength $n_{k,\ell}$ and reliability target ε_k , the finite-sub-blocklength rate is

$$R_{k,\ell}^{(m)} = C_{k,\ell}^{(m)} - \sqrt{\frac{V_{k,\ell}^{(m)}}{n_{k,\ell}}} Q^{-1}(\varepsilon_k). \quad (5)$$

If $b_{k,\ell}$ information bits are sent over $n_{k,\ell}$ channel uses, feasibility requires

$$\frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(m)}. \quad (6)$$

Multiplying both sides by $n_{k,\ell}$ yields

$$b_{k,\ell} \leq n_{k,\ell} R_{k,\ell}^{(m)}. \quad (7)$$

Since the service variable is integer-valued, the largest feasible blockwise service is

$$b_{k,\ell}^{\max,(m)} = \left\lfloor n_{k,\ell} R_{k,\ell}^{(m)} \right\rfloor. \quad (8)$$

Equation (8) is central throughout the report: it converts a continuous finite-sub-blocklength rate into an integer blockwise service limit.

3 Communication Scenarios

Before separating uplink and downlink, it is useful to define the two traffic interpretations used throughout the report. These scenarios determine what each block is asked to transmit, how latency is accumulated across blocks, and how unserved bits are interpreted.

3.1 Payload Communication

In payload-completion mode, each user begins with a finite payload B_k . The scheduler stops only when this payload is fully drained. The number of bits to send in block ℓ is the current remainder:

$$b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}. \quad (9)$$

After the block is committed, the remainder is updated as

$$B_{k,\text{rem}}^{(\ell+1)} = B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}. \quad (10)$$

Thus, payload completion is a workload-draining problem: every decision is made with respect to the residual bits that still have to be delivered.

3.2 Continuous Communication (Fixed Bits per Block)

In continuous communication mode, the transmission length L (total no. of sub-blocks) is known in advance and the blockwise targets are specified externally:

$$b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}, \quad \ell = 1, \dots, L. \quad (11)$$

The unmet service in a block is

$$u_{k,\ell} = \bar{b}_{k,\ell} - b_{k,\ell}, \quad u_{k,\ell} \geq 0, \quad (12)$$

which is recorded as an incomplete transmission of bits rather than being carried over to next sub-block. Thus, fixed continuous communication is a prescribed-demand problem: each block arrives with an externally given target.

3.3 Uplink Payload

For uplink payload completion, each user evolves independently according to its own remainder recursion. User k creates a new block only if its current block cannot finish the entire remaining payload. Thus, the block count L_k is an output of the optimization.

3.4 Uplink Continuous Communication

For uplink fixed continuous communication, the the block count L_k is set in advance. Each epoch has a target $\bar{b}_{k,\ell}$, and the main question is how much of that target can be supported within the coherence and power limits of the user at that epoch.

3.5 Downlink Payload

For downlink payload completion, the active-user set changes with time:

$$\mathcal{A}_\ell = \{k \in \mathcal{K} : B_{k,\text{rem}}^{(\ell)} > 0\}. \quad (13)$$

As users finish their payloads they leave the active set, which changes both the interference structure and the feasible rate region of the remaining users.

3.6 Downlink Continuous Communication

For downlink fixed continuous communication, the the block count L_k is prescribed and every block carries a target vector $\{\bar{b}_{k,\ell}\}_{k \in \mathcal{K}}$. The active-user set is determined by the users that are scheduled in that block. If the coupled downlink block cannot support a user's requested service, the user may receive zero service or partial service in that block, and this outcome is recorded.

4 Uplink Formulation

4.1 Effective Uplink Signal Model

In the uplink, user k in block ℓ is represented through the effective MIMO relation

$$\mathbf{y}_{k,\ell}^{(\text{UL})} = \mathbf{H}_{k,\ell}^{(\text{UL})} \mathbf{F}_{k,\ell}^{(\text{UL})} \mathbf{s}_{k,\ell} + \mathbf{z}_{k,\ell}^{(\text{UL})}, \quad (14)$$

where

$$\mathbf{H}_{k,\ell}^{(\text{UL})} \in \mathbb{C}^{N_{r,k} \times N_{t,k}}, \quad \mathbf{F}_{k,\ell}^{(\text{UL})} \in \mathbb{C}^{N_{t,k} \times d_k}, \quad (15)$$

$\mathbf{s}_{k,\ell}$ is the transmitted symbol vector with identity covariance, and $\mathbf{z}_{k,\ell}^{(\text{UL})}$ is an AWGN vector. Hence its covariance matrix is

$$\boldsymbol{\Sigma}_{k,\ell}^{(\text{UL})} := \sigma_k^2 \mathbf{I}. \quad (16)$$

The desired-signal covariance is

$$\mathbf{S}_{k,\ell}^{(\text{UL})} = \mathbf{H}_{k,\ell}^{(\text{UL})} \mathbf{F}_{k,\ell}^{(\text{UL})} (\mathbf{F}_{k,\ell}^{(\text{UL})})^H (\mathbf{H}_{k,\ell}^{(\text{UL})})^H, \quad (17)$$

and the whitened matrix becomes

$$\mathbf{A}_{k,\ell}^{(\text{UL})} = (\boldsymbol{\Sigma}_{k,\ell}^{(\text{UL})})^{-1/2} \mathbf{S}_{k,\ell}^{(\text{UL})} (\boldsymbol{\Sigma}_{k,\ell}^{(\text{UL})})^{-1/2}. \quad (18)$$

Substituting (18) into (3)–(5) gives $C_{k,\ell}^{(\text{UL})}$, $V_{k,\ell}^{(\text{UL})}$, and $R_{k,\ell}^{(\text{UL})}$.

4.2 Single-Block Uplink Design Problem

Suppose block ℓ of user k is asked to serve $b_{k,\ell}^{\text{req}}$ bits. In payload completion, $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$; in fixed continuous communication, $b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}$. The objective is the total latency of user k , namely $\tau_k = \frac{1}{f_{s,k}} \sum_{\ell'} n_{k,\ell'}$. When block ℓ is being designed, all terms except the current block are already fixed or deferred to the remaining schedule, so the blockwise design is written in the total-latency form

$$\min_{n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{UL})}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell'} n_{k,\ell'} \quad (19a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}, \quad (19b)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad (19c)$$

$$\|\mathbf{F}_{k,\ell}^{(\text{UL})}\|_F^2 \leq P_k. \quad (19d)$$

4.3 Uplink Payload-Completion Formulation

In payload completion mode, the number of blocks required by user k is not known in advance. Let L_k denote the number of blocks eventually used by user k . This terminal block index is induced by the resulting optimization and is therefore an output of the optimization rather than an independent optimization parameter. The uplink payload problem is

$$\min_{\{b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{UL})}\}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell=1}^{L_k} n_{k,\ell} \quad (20a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}, \quad \forall k, \ell, \quad (20b)$$

$$\sum_{\ell=1}^{L_k} b_{k,\ell} = B_k, \quad \forall k, \quad (20c)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k, \ell, \quad (20d)$$

$$\|\mathbf{F}_{k,\ell}^{(\text{UL})}\|_F^2 \leq P_k, \quad \forall k, \ell. \quad (20e)$$

The remaining-bits recursion is

$$B_{k,\text{rem}}^{(1)} = B_k, \quad B_{k,\text{rem}}^{(\ell+1)} = B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}, \quad (21)$$

and the scheduler terminates once $B_{k,\text{rem}}^{(\ell)} = 0$.

4.4 Uplink Fixed Continuous Communication Formulation

In fixed continuous communication mode, the block count L_k is prescribed in advance, and each block carries a target $\bar{b}_{k,\ell}$. Here $b_{k,\ell}$ denotes the transmitted bits of the block, whereas the design variables are the sub-blocklength and precoder. The problem becomes

$$\min_{\{n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{UL})}\}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell=1}^L n_{k,\ell} \quad (22a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}, \quad \forall k, \ell, \quad (22b)$$

$$b_{k,\ell} \in \{1, \dots, \bar{b}_{k,\ell}\}, \quad \forall k, \ell, \quad (22c)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k, \ell, \quad (22d)$$

$$\|\mathbf{F}_{k,\ell}^{(\text{UL})}\|_F^2 \leq P_k, \quad \forall k, \ell. \quad (22e)$$

The key difference from payload completion is that the block count L_k is fixed and blockwise shortfall $\bar{b}_{k,\ell} - b_{k,\ell}$ is recorded explicitly rather than being carried forward as an unscheduled residual payload.

5 Downlink Formulation

5.1 Interference-Coupled Downlink Signal Model

In the downlink, all active users in block ℓ are transmitted simultaneously. The received signal of user k is

$$\mathbf{y}_{k,\ell}^{(\text{DL})} = \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{k,\ell}^{(\text{DL})} \mathbf{s}_{k,\ell} + \sum_{j \in \mathcal{A}_\ell \setminus \{k\}} \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{j,\ell}^{(\text{DL})} \mathbf{s}_{j,\ell} + \mathbf{z}_{k,\ell}^{(\text{DL})}, \quad (23)$$

where

$$\mathbf{H}_{k,\ell}^{(\text{DL})} \in \mathbb{C}^{N_{r,k} \times N_b}, \quad \mathbf{F}_{k,\ell}^{(\text{DL})} \in \mathbb{C}^{N_b \times d_k}, \quad (24)$$

and $\mathbf{z}_{k,\ell}^{(\text{DL})}$ is receiver noise.

The desired-signal covariance of user k is

$$\mathbf{S}_{k,\ell}^{(\text{DL})} = \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{k,\ell}^{(\text{DL})} (\mathbf{F}_{k,\ell}^{(\text{DL})})^H (\mathbf{H}_{k,\ell}^{(\text{DL})})^H. \quad (25)$$

Its interference-plus-noise covariance is

$$\boldsymbol{\Sigma}_{k,\ell}^{(\text{DL})} = \sigma_k^2 \mathbf{I} + \sum_{j \in \mathcal{A}_\ell \setminus \{k\}} \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{j,\ell}^{(\text{DL})} (\mathbf{F}_{j,\ell}^{(\text{DL})})^H (\mathbf{H}_{k,\ell}^{(\text{DL})})^H. \quad (26)$$

Hence the downlink whitened matrix is

$$\mathbf{A}_{k,\ell}^{(\text{DL})} = (\boldsymbol{\Sigma}_{k,\ell}^{(\text{DL})})^{-1/2} \mathbf{S}_{k,\ell}^{(\text{DL})} (\boldsymbol{\Sigma}_{k,\ell}^{(\text{DL})})^{-1/2}, \quad (27)$$

which leads directly to $C_{k,\ell}^{(\text{DL})}$, $V_{k,\ell}^{(\text{DL})}$, and $R_{k,\ell}^{(\text{DL})}$ through (3)–(5).

5.2 Downlink Design Problem

Let $\mathcal{A}_\ell$ be the active-user set in block ℓ . For each $k \in \mathcal{A}_\ell$, let $b_{k,\ell}^{\text{req}}$ denote the requested bits to be transmitted of the current block, where $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$ in payload completion and $b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}$ in fixed continuous communication. The network objective remains the total latency $\tau_{\text{sum}} = \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell'} n_{k,\ell'}$. During block ℓ , only the active-user sub-blocklengths of the current block are free, so the one-block downlink problem is written as

$$\min_{\{n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{DL})}\}_{k \in \mathcal{A}_\ell}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell'} n_{k,\ell'} \quad (28a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{DL})}, \quad \forall k \in \mathcal{A}_\ell, \quad (28b)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k \in \mathcal{A}_\ell, \quad (28c)$$

$$\sum_{k \in \mathcal{A}_\ell} \left\| \mathbf{F}_{k,\ell}^{(\text{DL})} \right\|_F^2 \leq P_B. \quad (28d)$$

The complication relative to the uplink is immediate: $R_{k,\ell}^{(\text{DL})}$ depends on the entire active-user beam set through (26). Thus, even if only per User's precoder is changed, the feasible rates of all other active users are altered as well. Hence the current-block decision cannot be separated from the coupled active-user state of the block.

5.3 Downlink Payload-Completion Formulation

In payload completion mode, the active-user set is induced by the remaining payload:

$$\mathcal{A}_\ell = \left\{ k \in \mathcal{K} : B_{k,\text{rem}}^{(\ell)} > 0 \right\}. \quad (29)$$

The network-level problem is

$$\min_{\{b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{DL})}\}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell} n_{k,\ell} \quad (30a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{DL})}, \quad \forall k, \ell, \quad (30b)$$

$$\sum_{\ell} b_{k,\ell} = B_k, \quad \forall k, \quad (30c)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k, \ell, \quad (30d)$$

$$\sum_{k \in \mathcal{A}_\ell} \left\| \mathbf{F}_{k,\ell}^{(\text{DL})} \right\|_F^2 \leq P_B, \quad \forall \ell. \quad (30e)$$

The same remaining-bits recursion as in (21) applies, but now the block service of one user depends on the simultaneous service decisions of the others.

5.4 Downlink Fixed Continuous Communication Formulation

In fixed continuous communication mode, the the block count L_k is prescribed and the blockwise transmission bits targets are fixed. Again, $b_{k,\ell}$ denotes the transmitted bits, whereas the design variables are the sub-blocklengths and precoders:

$$\min_{\{n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{DL})}\}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell=1}^L n_{k,\ell} \quad (31a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{DL})}, \quad \forall k, \ell, \quad (31b)$$

$$0 \leq b_{k,\ell} \leq \bar{b}_{k,\ell}, \quad \forall k, \ell, \quad (31c)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k, \ell, \quad (31d)$$

$$\sum_{k \in \mathcal{A}_\ell} \left\| \mathbf{F}_{k,\ell}^{(\text{DL})} \right\|_F^2 \leq P_B, \quad \forall \ell. \quad (31e)$$

The difference from payload completion is again conceptual: the demand sequence is externally prescribed, so any shortfall $\bar{b}_{k,\ell} - b_{k,\ell}$ is treated as unmet service in that block rather than as a residual workload to be automatically rolled forward.

6 Training Only Method

6.1 Common Primal-Dual Structure

The *Training Only Method* performs optimization directly on the current channel realization. There is no offline learning stage. Instead, each decision block is solved online through a constrained primal-dual update. In payload completion, $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$, whereas in fixed continuous communication, $b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}$.

For uplink block (k, ℓ) with requested service $b_{k,\ell}^{\text{req}}$, define the rate constraint residual

$$g_{k,\ell}^{(\text{UL},r)} = \frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} - R_{k,\ell}^{(\text{UL})}, \quad (32)$$

and the power residual

$$g_{k,\ell}^{(\text{UL},p)} = \left\| \mathbf{F}_{k,\ell}^{(\text{UL})} \right\|_F^2 - P_k. \quad (33)$$

For the downlink block, define

$$g_{k,\ell}^{(\text{DL},r)} = \frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} - R_{k,\ell}^{(\text{DL})}, \quad k \in \mathcal{A}_\ell, \quad (34)$$

and the common block-power residual

$$g_\ell^{(\text{DL},p)} = \sum_{k \in \mathcal{A}_\ell} \left\| \mathbf{F}_{k,\ell}^{(\text{DL})} \right\|_F^2 - P_B. \quad (35)$$

The blockwise Lagrangian is then formed by combining a rate-maximizing primal objective with nonnegative dual penalties. In uplink,

$$\mathcal{L}_{k,\ell}^{(\text{UL})} = -R_{k,\ell}^{(\text{UL})} + \lambda_{k,\ell}^{(\text{UL},r)} [g_{k,\ell}^{(\text{UL},r)}]_+ + \lambda_{k,\ell}^{(\text{UL},p)} [g_{k,\ell}^{(\text{UL},p)}]_+, \quad (36)$$

while in downlink,

$$\mathcal{L}_\ell^{(\text{DL})} = - \sum_{k \in \mathcal{A}_\ell} R_{k,\ell}^{(\text{DL})} + \sum_{k \in \mathcal{A}_\ell} \lambda_{k,\ell}^{(\text{DL},r)} [g_{k,\ell}^{(\text{DL},r)}]_+ + \lambda_\ell^{(\text{DL},p)} [g_\ell^{(\text{DL},p)}]_+. \quad (37)$$

The primal variables are updated to reduce the Lagrangian, and the dual variables are updated to penalize persistent infeasibility. To monitor the solve, the method tracks three KKT residuals.

For uplink block (k, ℓ) ,

$$r_{p,k,\ell}^{(\text{UL})} = \max \left\{ [g_{k,\ell}^{(\text{UL},r)}]_+, [g_{k,\ell}^{(\text{UL},p)}]_+ \right\}, \quad (38)$$

$$r_{c,k,\ell}^{(\text{UL})} = \max \left\{ \lambda_{k,\ell}^{(\text{UL},r)} [g_{k,\ell}^{(\text{UL},r)}]_+, \lambda_{k,\ell}^{(\text{UL},p)} [g_{k,\ell}^{(\text{UL},p)}]_+ \right\}, \quad (39)$$

$$r_{s,k,\ell}^{(\text{UL})} = \frac{\left\| \mathbf{F}_{k,\ell}^{(\text{UL}), (t)} - \mathbf{F}_{k,\ell}^{(\text{UL}), (t-1)} \right\|_F}{\max \left\{ \left\| \mathbf{F}_{k,\ell}^{(\text{UL}), (t-1)} \right\|_F, \delta_0 \right\}}, \quad (40)$$

where $\delta_0 > 0$ is a small safeguard constant.

For downlink block ℓ ,

$$r_{p,\ell}^{(\text{DL})} = \max \left\{ \max_{k \in \mathcal{A}_\ell} [g_{k,\ell}^{(\text{DL},r)}]_+, [g_\ell^{(\text{DL},p)}]_+ \right\}, \quad (41)$$

$$r_{c,\ell}^{(\text{DL})} = \max \left\{ \max_{k \in \mathcal{A}_\ell} \lambda_{k,\ell}^{(\text{DL},r)} [g_{k,\ell}^{(\text{DL},r)}]_+, \lambda_\ell^{(\text{DL},p)} [g_\ell^{(\text{DL},p)}]_+ \right\}, \quad (42)$$

$$r_{s,\ell}^{(\text{DL})} = \max_{k \in \mathcal{A}_\ell} \frac{\left\| \mathbf{F}_{k,\ell}^{(\text{DL}), (t)} - \mathbf{F}_{k,\ell}^{(\text{DL}), (t-1)} \right\|_F}{\max \left\{ \left\| \mathbf{F}_{k,\ell}^{(\text{DL}), (t-1)} \right\|_F, \delta_0 \right\}}, \quad (43)$$

The online solve is declared converged when

$$r_p \leq \epsilon_p, \quad r_c \leq \epsilon_c, \quad r_s \leq \epsilon_s, \quad (44)$$

and it is declared stationary infeasible when

$$r_s \leq \epsilon_s \quad \text{but} \quad r_p > \epsilon_p. \quad (45)$$

6.2 Uplink Training Only Method

The uplink training-only method is fundamentally sequential, but it follows the same *converge first, allocate second, reduce third* logic as the downlink method. For each user-local block, the solver first converges the full-sub-blocklength uplink precoder at $n_{k,\ell} = T_k$, then computes the supported bits, and only then attempts sub-blocklength reduction if the entire current request has already been met. In payload completion mode, the full-block service is

$$b_{k,\ell}^{(T)} = \min \left\{ B_{k,\text{rem}}^{(\ell)}, \left\lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \right\rfloor \right\}. \quad (46)$$

In fixed continuous communication mode, it is

$$b_{k,\ell}^{(T)} := \min \left\{ \bar{b}_{k,\ell}, \left\lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \right\rfloor \right\}. \quad (47)$$

sub-blocklength reduction is attempted only after the current block request has been fully met. In payload completion, this means $b_{k,\ell}^{(T)} = B_{k,\text{rem}}^{(\ell)}$. In fixed continuous communication, this means $b_{k,\ell}^{(T)} = \bar{b}_{k,\ell}$. If the current request is only partially served, the block is committed at $n_{k,\ell} = T_k$ and the method moves to the next request without trying to reduce sub-blocklength.

Algorithm 1 Training-Only Uplink User Solver

Require: Scenario mode, user k , payload B_k or prescribed targets $\{\bar{b}_{k,\ell}\}$, tolerances $\epsilon_p, \epsilon_c, \epsilon_s$

Ensure: Block sequence $\{(b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{UL})})\}$

- 1: **if** payload-completion mode **then**
- 2: Set $B_{k,\text{rem}}^{(1)} \leftarrow B_k$
- 3: Initialize $\ell \leftarrow 1$
- 4: **while** a new block request exists **do**
- 5: **if** payload-completion mode **then**
- 6: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$
- 7: **else**
- 8: Set $b_{k,\ell}^{\text{req}} \leftarrow \bar{b}_{k,\ell}$
- 9: Set $n_{k,\ell} \leftarrow T_k$ and initialize $\mathbf{F}_{k,\ell}^{(\text{UL})}$, $\lambda_{k,\ell}^{(\text{UL},r)}$, and $\lambda_{k,\ell}^{(\text{UL},p)}$
- 10: **repeat**
- 11: Update $\mathbf{F}_{k,\ell}^{(\text{UL})}$ using the uplink Lagrangian (36)
- 12: Update $\lambda_{k,\ell}^{(\text{UL},r)}$ and $\lambda_{k,\ell}^{(\text{UL},p)}$
- 13: Measure $r_{p,k,\ell}^{(\text{UL})}$, $r_{c,k,\ell}^{(\text{UL})}$, and $r_{s,k,\ell}^{(\text{UL})}$ from (38)–(40)
- 14: **until** the uplink solve satisfies (44) or (45)
- 15: Restore the best feasible state if one exists; otherwise restore the best minimum-violation state
- 16: Compute $b_{k,\ell}^{(T)} = \min\{b_{k,\ell}^{\text{req}}, \lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \rfloor\}$
- 17: **if** $b_{k,\ell}^{(T)} = 0$ **then**
- 18: Set $b_{k,\ell} \leftarrow 0$ and keep $n_{k,\ell} = T_k$
- 19: **if** payload-completion mode **then**
- 20: Declare the remaining payload infeasible under the current state and stop
- 21: **else if** $b_{k,\ell}^{(T)} < b_{k,\ell}^{\text{req}}$ **then**
- 22: Commit $b_{k,\ell} = b_{k,\ell}^{(T)}$ and set $n_{k,\ell} = T_k$
- 23: Do not reduce sub-blocklength in this block
- 24: **else**
- 25: Set $b_{k,\ell} = b_{k,\ell}^{\text{req}}$
- 26: **while** a smaller candidate sub-blocklength remains feasible after re-optimization **do**
- 27: Tentatively replace $n_{k,\ell}$ by the next smaller candidate
- 28: Re-solve the candidate state with the same primal-dual loop above
- 29: Restore the best feasible candidate state if one exists; otherwise restore the previously accepted state
- 30: **if** $\frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}$ and $\|\mathbf{F}_{k,\ell}^{(\text{UL})}\|_F^2 \leq P_k$ **then**
- 31: Accept the candidate and continue
- 32: **else**
- 33: Restore the previous accepted sub-blocklength and stop reduction
- 34: Commit the smallest accepted $n_{k,\ell}$
- 35: **if** payload-completion mode **then**
- 36: Update $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$
- 37: **else**
- 38: Record the shortfall $u_{k,\ell} \leftarrow \bar{b}_{k,\ell} - b_{k,\ell}$
- 39: $\ell \leftarrow \ell + 1$

Algorithm 1 solves both uplink scenarios. In payload completion mode, the next request is the remaining payload. In fixed continuous communication mode, the next request is simply the

next prescribed target.

6.3 Downlink Training Only Method

In the downlink, a whole active-user block must be solved jointly. The same three-phase logic is applied, but now on a coupled active-user set. The method first converges the joint full-sub-blocklength precoder at the coherence limits $\{T_k\}_{k \in \mathcal{A}_\ell}$, then computes the supported bits of all requested users, removes users with zero supported service, and only then reduces sub-blocklength for the users whose full current requests are already supported. This produces the full-block supported service

$$b_{k,\ell}^{(T)} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor T_k R_{k,\ell}^{(\text{DL})}(T_k) \right\rfloor \right\}, \quad k \in \mathcal{A}_\ell. \quad (48)$$

Users with $b_{k,\ell}^{(T)} = 0$ are removed from transmission in that block. For users that satisfy $b_{k,\ell}^{(T)} = b_{k,\ell}^{\text{req}}$, the method tries to reduce sub-blocklength while keeping the already committed bits fixed. Since the block is coupled, a candidate reduction may invalidate another user's committed service. The method therefore re-optimizes the affected active-user subset whenever necessary and accepts a candidate reduction only if all committed bits remain feasible.

Algorithm 2 Training-Only Downlink Block Solver

Require: Scenario mode, active-user set $\mathcal{A}_\ell$, remaining payloads $\{B_{k,\text{rem}}^{(\ell)}\}_{k \in \mathcal{A}_\ell}$ or targets $\{\bar{b}_{k,\ell}\}_{k \in \mathcal{A}_\ell}$, tolerances $\epsilon_p, \epsilon_c, \epsilon_s$

Ensure: Block plan $\{(b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{DL})})\}_{k \in \mathcal{A}_\ell}$

- 1: Store the requested-user set as $\mathcal{A}_\ell^{\text{req}} \leftarrow \mathcal{A}_\ell$
- 2: **for** each $k \in \mathcal{A}_\ell^{\text{req}}$ **do**
- 3: **if** payload-completion mode **then**
- 4: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$
- 5: **else**
- 6: Set $b_{k,\ell}^{\text{req}} \leftarrow \bar{b}_{k,\ell}$
- 7: Set $n_{k,\ell} \leftarrow T_k$
- 8: Initialize the active-user precoders and multipliers
- 9: **repeat**
- 10: Update the active-user precoders using the block Lagrangian (37)
- 11: Update the multipliers $\{\lambda_{k,\ell}^{(\text{DL},r)}\}$ and $\lambda_\ell^{(\text{DL},p)}$
- 12: Measure $r_{p,\ell}^{(\text{DL})}$, $r_{c,\ell}^{(\text{DL})}$, and $r_{s,\ell}^{(\text{DL})}$ from (41)–(43)
- 13: **until** the downlink solve satisfies (44) or (45)
- 14: Restore the best feasible block state if one exists; otherwise restore the best minimum-violation state
- 15: **for** each active user $k \in \mathcal{A}_\ell$ **do**
- 16: Compute $b_{k,\ell}^{(T)}$ from (48)
- 17: Set $b_{k,\ell} \leftarrow 0$ for each user satisfying $b_{k,\ell}^{(T)} = 0$
- 18: Remove users with zero supported service from the transmit set of the block
- 19: **if** the transmit set changes **then**
- 20: Re-solve the reduced active-user block with the same primal-dual loop above
- 21: Restore the best feasible reduced-set state if one exists; otherwise restore the previously accepted state
- 22: **for** each remaining active user $k \in \mathcal{A}_\ell$ **do**
- 23: Recompute $b_{k,\ell}^{(T)}$ from (48)
- 24: **for** each user satisfying $b_{k,\ell}^{(T)} = b_{k,\ell}^{\text{req}}$ **do**
- 25: **while** a smaller candidate sub-blocklength preserves all already committed bits **do**
- 26: Tentatively decrease $n_{k,\ell}$ while holding the other active-user sub-blocklengths fixed
- 27: Re-optimize the affected active-user subset with the same primal-dual loop above
- 28: Restore the best feasible candidate state if one exists; otherwise restore the previously accepted state
- 29: **if** $\frac{b_{j,\ell}^{\text{req}}}{n_{j,\ell}} \leq R_{j,\ell}^{(\text{DL})}$ for all active users $j \in \mathcal{A}_\ell$ and $\sum_{j \in \mathcal{A}_\ell} \|\mathbf{F}_{j,\ell}^{(\text{DL})}\|_F^2 \leq P_B$ **then**
- 30: Accept the candidate and continue
- 31: **else**
- 32: Restore the previous accepted sub-blocklength of user k and stop reducing that user
- 33: **for** each requested user $k \in \mathcal{A}_\ell^{\text{req}}$ **do**
- 34: **if** $k \in \mathcal{A}_\ell$ **then**
- 35: Set $b_{k,\ell} \leftarrow \min\left\{b_{k,\ell}^{\text{req}}, \left\lfloor n_{k,\ell} R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \right\rfloor\right\}$
- 36: **if** payload-completion mode **then**
- 37: Update $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$
- 38: **else**
- 39: Record the shortfall $u_{k,\ell} \leftarrow \bar{b}_{k,\ell} - b_{k,\ell}$
- 40: Commit the final joint block plan

The training-only downlink method is therefore a *converge first, allocate second, reduce third* procedure. This ordering is essential because the finite-sub-blocklength rate of each user depends on the full active-user beam set.

7 Training + Testing Method

7.1 Method Principle

The *Training + Testing Method* separates optimization into two stages. During training, a precoder mapping is learned from a collection of queries produced by the same finite-sub-blocklength scheduling logic that will later be used at test time. During testing, the online optimization step is replaced by model inference, followed by the same feasibility checks and the same sub-blocklength-reduction logic.

The central training object is a query. In general form, a query $q \in \mathcal{Q}$ contains

$$q = (\text{channel state, active-user state, candidate sub-blocklength state, bits-to-send state, covariance state, reliability target}). \quad (49)$$

Thus, a query is simply one scheduling state at which the model is asked to predict a feasible finite-sub-blocklength transmission strategy. In the present formulation, the training loss uses *uniform-per-query* averaging, so every stored query contributes equally to the empirical objective.

7.2 Uplink Training + Testing Method

7.2.1 Training stage

For uplink user k , a query at block ℓ has the form

$$q_{k,\ell}^{(\text{UL})} = (\mathbf{H}_{k,\ell}^{(\text{UL})}, \mathbf{\Sigma}_{k,\ell}^{(\text{UL})}, n, b_{k,\ell}^{\text{req}}, \varepsilon_k), \quad (50)$$

where n is the candidate sub-blocklength being queried and $b_{k,\ell}^{\text{req}}$ is the number of bits to send in that state. The query set $\mathcal{Q}_k^{(\text{UL})}$ is built by running the same causal block logic used by the scheduler itself and recording every visited candidate state. In payload completion, the number of bits to send is $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$. In fixed continuous communication, it is $b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}$.

Let θ_k denote the trainable parameters of the uplink model for user k . The uniform-per-query training objective is

$$\mathcal{J}_k^{(\text{UL})}(\theta_k) = \frac{1}{|\mathcal{Q}_k^{(\text{UL})}|} \sum_{q \in \mathcal{Q}_k^{(\text{UL})}} \left(-R(q; \theta_k) + \lambda_k^{(r)} \left[\frac{b^{\text{req}}(q)}{n(q)} - R(q; \theta_k) \right]_+ + \lambda_k^{(p)} [P(q, \theta_k) - P_k]_+ \right), \quad (51)$$

where $R(q; \theta_k)$ is the finite-sub-blocklength rate induced by the predicted uplink precoder, $n(q)$ is the queried sub-blocklength, $b^{\text{req}}(q)$ is the requested bit count encoded in query q , and $P(q, \theta_k)$ is the transmit power induced by the predicted precoder. The coefficient $\lambda_k^{(r)}$ penalizes rate shortfall relative to the requested service, whereas $\lambda_k^{(p)}$ penalizes power-budget violation.

Algorithm 3 Uplink Training Stage

Require: Scenario mode, training episodes for uplink user k

Ensure: Query set $\mathcal{Q}_k^{(\text{UL})}$ and trained parameters θ_k

- 1: Initialize $\mathcal{Q}_k^{(\text{UL})} \leftarrow \emptyset$
- 2: **for** each training episode **do**
- 3: **if** payload-completion mode **then**
- 4: Initialize the episode remainder state from the episode payload
- 5: **for** each visited block ℓ of user k **do**
- 6: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ in payload completion, or $b_{k,\ell}^{\text{req}} \leftarrow \bar{b}_{k,\ell}$ in fixed continuous communication
- 7: Set the first candidate sub-blocklength to $n \leftarrow T_k$
- 8: **while** n is under consideration **do**
- 9: Form the query $q_{k,\ell}^{(\text{UL})}$ in (50) and add it to $\mathcal{Q}_k^{(\text{UL})}$
- 10: Evaluate the current state and compute

$$b_{k,\ell}^{(n)} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor n R_{k,\ell}^{(\text{UL})}(n) \right\rfloor \right\}$$

- 11: **if** $b_{k,\ell}^{(n)} < b_{k,\ell}^{\text{req}}$ **then**
 - 12: Stop collecting additional queries for this sub-block
 - 13: **else**
 - 14: Replace n by the next smaller candidate sub-blocklength
 - 15: **if** payload-completion mode **then**
 - 16: Advance the episode remainder according to the committed service of the generating schedule
 - 17: **else**
 - 18: Advance to the next prescribed target of the generating schedule
 - 19: Train θ_k by minimizing (51)
-

7.2.2 Testing stage

At test time, the trained model replaces the online uplink solve. The block is first examined at the full available sub-blocklength. If the full block does not support the complete request, the scheduler commits only the supported service at $n = T_k$. If the full block already supports the entire request, the sub-blocklength is reduced and the model is queried again at smaller candidate sub-blocklengths until the smallest feasible one is identified.

Algorithm 4 Uplink Testing Stage

Require: Scenario mode, trained uplink model θ_k , user k , payload B_k or prescribed targets $\{\bar{b}_{k,\ell}\}$

Ensure: Final uplink block plan

1: **if** payload-completion mode **then**

2: Set $B_{k,\text{rem}}^{(1)} \leftarrow B_k$

3: **for** each visited block ℓ **do**

4: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ in payload completion, or $b_{k,\ell}^{\text{req}} \leftarrow \bar{b}_{k,\ell}$ in fixed continuous communication

5: Infer the uplink precoder at $n = T_k$

6: Compute the full-block supported service

$$b_{k,\ell}^{(T)} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \right\rfloor \right\}$$

7: **if** $b_{k,\ell}^{(T)} < b_{k,\ell}^{\text{req}}$ **then**

8: Commit $n_{k,\ell} = T_k$ and $b_{k,\ell} = b_{k,\ell}^{(T)}$

9: **else**

10: Set $b_{k,\ell} = b_{k,\ell}^{\text{req}}$

11: Set $n_{k,\ell} \leftarrow T_k$

12: **while** a smaller candidate sub-blocklength exists **do**

13: Tentatively replace $n_{k,\ell}$ by the next smaller candidate

14: Infer the uplink precoder for the updated query

15: **if** $\frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}(n_{k,\ell})$ and $P(q_{k,\ell}^{(\text{UL})}, \theta_k) \leq P_k$ **then**

16: Accept the reduction and continue

17: **else**

18: Restore the previous sub-blocklength and stop reduction

19: **if** payload-completion mode **then**

20: Update $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$

21: **else**

22: Record the shortfall $u_{k,\ell} \leftarrow \bar{b}_{k,\ell} - b_{k,\ell}$

7.3 Downlink Training + Testing Method

7.3.1 Training stage

The downlink training state is joint across the active users of a block. A query for block ℓ can be written as

$$q_\ell^{(\text{DL})} = (\{\mathbf{H}_{k,\ell}^{(\text{DL})}\}_{k \in \mathcal{A}_\ell}, \mathcal{A}_\ell, \{n_{k,\ell}\}_{k \in \mathcal{A}_\ell}, \{b_{k,\ell}^{\text{req}}\}_{k \in \mathcal{A}_\ell}, \{\Sigma_{k,\ell}^{(\text{DL})}\}_{k \in \mathcal{A}_\ell}, \{\varepsilon_k\}_{k \in \mathcal{A}_\ell}), \quad (52)$$

where $\mathcal{A}_\ell$ is the active-user set of the block. The query set $\mathcal{Q}^{(\text{DL})}$ is built causally by running the same joint scheduling logic used at test time and recording every visited joint state. In payload completion, the number of bits to send is $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$. In fixed continuous communication, it is $b_{k,\ell}^{\text{req}} = \bar{b}_{k,\ell}$.

Let $\theta = \{\theta_k\}_{k=1}^K$ denote the trainable parameters of the downlink model collection. The

uniform-per-query training objective is

$$\begin{aligned} \mathcal{J}^{(\text{DL})}(\theta) = \frac{1}{|\mathcal{Q}^{(\text{DL})}|} \sum_{q \in \mathcal{Q}^{(\text{DL})}} \bigg(& - \sum_{k \in \mathcal{A}(q)} R_k(q; \theta) \\ & + \sum_{k \in \mathcal{A}(q)} \lambda_k^{(r)} \left[\frac{b_k^{\text{req}}(q)}{n_k(q)} - R_k(q; \theta) \right]_+ \\ & + \lambda^{(p)} \left[\sum_{k \in \mathcal{A}(q)} P_k(q, \theta) - P_B \right]_+ \bigg). \end{aligned} \quad (53)$$

where $\mathcal{A}(q)$ is the active-user set encoded in query q , $n_k(q)$ is the sub-blocklength associated with active user k in that query, $b_k^{\text{req}}(q)$ is the number of bits to send for active user k , and $P_k(q, \theta)$ is the transmit power induced for active user k by the predicted downlink precoder. The penalty coefficients $\lambda_k^{(r)}$ and $\lambda^{(p)}$ enforce the rate constraints and the total base-station power constraint, respectively.

Algorithm 5 Downlink Training Stage

Require: Scenario mode and training episodes

Ensure: Query set $\mathcal{Q}^{(\text{DL})}$ and trained parameters θ

- 1: Initialize $\mathcal{Q}^{(\text{DL})} \leftarrow \emptyset$
 - 2: **for** each training episode **do**
 - 3: **for** each visited communication block ℓ **do**
 - 4: Determine the active-user set $\mathcal{A}_\ell$
 - 5: Store the queried-user set as $\mathcal{A}_\ell^{\text{req}} \leftarrow \mathcal{A}_\ell$
 - 6: **for** each $k \in \mathcal{A}_\ell$ **do**
 - 7: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ in payload completion, or $b_{k,\ell}^{\text{req}} \leftarrow \bar{b}_{k,\ell}$ in fixed continuous communication
 - 8: Initialize $n_{k,\ell} \leftarrow T_k$
 - 9: **while** the current joint sub-blocklength state is under consideration **do**
 - 10: Form the joint query $q_\ell^{(\text{DL})}$ in (52) and add it to $\mathcal{Q}^{(\text{DL})}$
 - 11: Evaluate the current joint state and compute, for each $k \in \mathcal{A}_\ell$,
 - $$b_{k,\ell}^{(\mathbf{n})} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lceil n_{k,\ell} R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \right\rceil \right\}$$
 - 12: Remove users with $b_{k,\ell}^{(\mathbf{n})} = 0$ from the transmit set if such users exist
 - 13: **if** no user satisfies $b_{k,\ell}^{(\mathbf{n})} = b_{k,\ell}^{\text{req}}$ **then**
 - 14: Stop collecting additional queries for this block
 - 15: **else**
 - 16: Decrease the candidate sub-blocklengths of users already satisfying $b_{k,\ell}^{(\mathbf{n})} = b_{k,\ell}^{\text{req}}$
 - 17: **if** payload-completion mode **then**
 - 18: Advance the episode remainders according to the committed service of the generating schedule
 - 19: **else**
 - 20: Advance to the next prescribed downlink block of the generating schedule
 - 21: Train θ by minimizing (53)
-

7.3.2 Testing stage

At test time, the learned downlink model is queried on the current active-user state of the block. The full joint sub-blocklength vector is evaluated first. Users with zero supported service are

removed from the transmit set, and joint sub-blocklength reduction is attempted only for users whose full requested service is already supported at the current state.

Algorithm 6 Downlink Testing Stage

Require: Scenario mode, trained downlink model θ , remaining payloads $\{B_{k,\text{rem}}^{(\ell)}\}$ or prescribed targets $\{\bar{b}_{k,\ell}\}$

Ensure: Final downlink block plan

- 1: **for** each communication block ℓ **do**
- 2: Determine the active-user set $\mathcal{A}_\ell$
- 3: Store the requested-user set as $\mathcal{A}_\ell^{\text{req}} \leftarrow \mathcal{A}_\ell$
- 4: **for** each $k \in \mathcal{A}_\ell$ **do**
- 5: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ in payload completion, or $b_{k,\ell}^{\text{req}} \leftarrow \bar{b}_{k,\ell}$ in fixed continuous communication
- 6: Initialize $n_{k,\ell} \leftarrow T_k$
- 7: Infer the joint downlink precoders at the full sub-blocklength vector
- 8: Compute, for each $k \in \mathcal{A}_\ell$,

$$b_{k,\ell}^{(T)} = \min\left\{b_{k,\ell}^{\text{req}}, \left\lfloor T_k R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \right\rfloor\right\}$$

- 9: Set $b_{k,\ell} \leftarrow 0$ for each user satisfying $b_{k,\ell}^{(T)} = 0$
 - 10: Remove users with $b_{k,\ell}^{(T)} = 0$ from the transmit set and re-infer the joint precoders if the active-user set changes
 - 11: **for** each user satisfying $b_{k,\ell}^{(T)} = b_{k,\ell}^{\text{req}}$ **do**
 - 12: **while** a smaller candidate sub-blocklength exists for that user **do**
 - 13: Tentatively decrease $n_{k,\ell}$ while holding the other active-user sub-blocklengths fixed
 - 14: Infer the joint downlink precoders for the updated state
 - 15: **if** $\frac{b_{j,\ell}^{\text{req}}}{n_{j,\ell}} \leq R_{j,\ell}^{(\text{DL})}(\mathbf{n}_\ell)$ for all active users j and $\sum_{j \in \mathcal{A}_\ell} P_j(q_\ell^{(\text{DL})}, \theta) \leq P_B$ **then**
 - 16: Accept the reduction
 - 17: **else**
 - 18: Restore the previous sub-blocklength of user k and stop reducing that user
 - 19: **for** each remaining active user $k \in \mathcal{A}_\ell$ **do**
 - 20: Set $b_{k,\ell} \leftarrow \min\left\{b_{k,\ell}^{\text{req}}, \left\lfloor n_{k,\ell} R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \right\rfloor\right\}$
 - 21: **for** each requested user $k \in \mathcal{A}_\ell^{\text{req}}$ **do**
 - 22: **if** payload-completion mode **then**
 - 23: Update $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$
 - 24: **else**
 - 25: Record the shortfall $u_{k,\ell} \leftarrow \bar{b}_{k,\ell} - b_{k,\ell}$
 - 26: Commit the final active-user set, sub-blocklengths, and transmitted bits of the block
-

8 Conclusion

The report has recast the current work into a theory-centered structure. The development proceeds from a common finite-sub-blocklength model and communication scenarios, to separate uplink and downlink formulations, and then to two solution frameworks.

The uplink problem is characterized by user-wise sequential block creation under finite-sub-blocklength feasibility. The downlink problem is characterized by a jointly coupled interference structure that requires block-level coordination before blockwise service can be committed.

The *Training Only Method* solves each decision state online through constrained primal-dual optimization and feasibility-guided sub-blocklength reduction. The *Training + Testing Method* replaces online optimization by learned precoder inference, but retains the same finite-sub-blocklength service logic through query-based offline training, feasibility checks, and causal sub-blocklength reduction.

Taken together, these formulations provide a complete theoretical description of the current uplink and downlink methodology without relying on implementation details or numerical experiment summaries.

References

- [1] Y. Polyanskiy, H. V. Poor, and S. Verdú, “Channel coding rate in the finite sub-blocklength regime,” *IEEE Transactions on Information Theory*, vol. 56, no. 5, pp. 2307–2359, 2010.